Microprocessor Control System Final Report

Household robot dog implementation

Group 4

Team members:
B12502027 CHENG-HSUAN LEE
B12502140 Li Yaocheng
B12502015 Li Zuiyu
B12701239 Zhuang Ziyi

Directory

1. Summary

This topic is based on the theme of "Realization of Household Robot Dogs", and a small four-legged robot dog with 8 degrees of freedom is designed and completed. The system uses ESP32 as the main controller and the PCA9685 servo motor drive module to control 8 MG90S servo motors and perform wireless operations through the mobile web interface. The Trot diagonal gait and 2-DOF inverse kinematics are introduced into the program, allowing the robot dog to complete actions such as standing, lying down, moving forward, retreating, waving, raising the head, raising the buttocks, and rocking back and forth.

During the implementation process, we made adjustments to address issues such as insufficient motor torque, slippage on the soles of the feet, unstable power supply, and shifting center of gravity, including improving the power supply structure, adjusting gait parameters, correcting the stance angle, and increasing sole friction. Finally, a low-cost quadruped robot platform with basic mobile

and interactive display functions is completed, while leaving room for expansion in the future by adding vision, sensors and smart interactive functions.

2. Implement the function and situation description of the finished product

The traditional 12-degree-of-freedom quadruped robot control system is extremely complex and has high hardware and maintenance costs, making it difficult to be widely used in the general consumer market or in daily small spaces. In order to deeply explore the dynamic walking gait, inverse kinematics and wireless microprocessor control system of multi-link mechanisms within a limited development cycle and budget, this topic designed and implemented a simplified version of a small quadruped robot dog with 8 degrees of freedom (quadruped configuration, each leg consisting of a thigh and calf 2-DOF linkage mechanism).

In terms of the overall electromechanical architecture, the dual-core 32-bit processor ESP32 is used as the main control core, responsible for the inverse kinematics calculation, gait planning and Wi-Fi web server operation of the entire machine. Since 8 servo motors need to be controlled at the same time, the system connects the PCA9685 servo control board through the I2C bus as a PWM signal expansion module to generate a 50Hz control signal to reduce the burden on the main control chip and improve signal stability.

Taking into account the huge instantaneous current generated when multiple motors operate at the same time, this system adopts a protection design in which the power supply and control circuits are planned separately, but the two share a common ground. The system uses a UBEC 8A transformer module to provide the stable 5V power supply required by the motor; the control circuit is independently powered by a DC-DC Mini560 step-down module, which effectively isolates the voltage fluctuation caused by the motor when pumping load, completely avoiding the problem of restarting the main controller ESP32 or Wi-Fi control interruption.

2.1 Starting the engine and usage scenarios

As the world moves into a super-aging society and a declining birthrate, the proportion of elderly people living alone and children who need companionship is increasing year by year. According to international market research, the global companion robot market is growing explosively at a growth rate of approximately 17% to 25%. This type of robot is gradually transforming from a mere industrial vehicle and is widely used to alleviate the loneliness of the elderly, prevent dementia, and assist in the social and cognitive development of children.

However, the high-end companion robot dogs or industrial-grade quadruped robot systems currently on the market are extremely closed and expensive, making it difficult to spread them to ordinary

families. Based on this social demand and pain point, in this topic, we use low cost and high scalability as the starting point, using the ESP32 dual-core microprocessor and PCA9685 expansion module to design and implement a miniature four-legged robot dog. It is hoped that a "digital home pet" with realistic body language and interactive temperature can be created at a low cost.

Usage situation:

1. **Emotional sustenance and companionship for the elderly living alone:**
   For seniors who live alone for a long time, robot dogs can accompany them in the home environment in a pet-like manner, providing low-burden emotional support and reducing loneliness and social isolation.
2. **Interactive playmate for children:**
   The robot dog can serve as a safe indoor playmate for children after school. Through posture changes, such as stretching, waving, etc., it attracts children to interact. In the future, it can even be combined with education and voice functions to assist children's early development.
3. **Mobile IoT nodes for smart homes:**
   Because it's compact and non-destructive, it can easily maneuver around an apartment living room, around a table or under a sofa. As a connected vehicle, as long as relevant modules are mounted in the future, it will have the potential to accompany you and patrol at home.

2.2 Implement finished product features and existing functions

Corresponding to the "digital home pet" scenario, the following features have been implemented through web UI and algorithms:

1. **Cross-platform wireless web remote control interface:**
   ESP32 will create a dedicated Wi-Fi hotspot after powering on. Users do not need to download the App and can open the graphical control panel through the mobile browser, which greatly reduces the operating threshold for the elderly and children.
2. **State machine security protection and silent storage:**
   The system has built-in rigorous state machine logic, and the default is the lying down storage posture. You must click "Wake Up" on the UI to unlock it before you can move. This prevents accidental contact from causing the motor to instantaneously output power at the wrong angle and burn out, ensuring safe home operation.
3. **Realistic posture algorithm:**
   Import inverse kinematics to calculate the toe target trajectory. At present, functions such as "waving to say hello", "raising head/raising butt" and "swaying back and forth" have been implemented, making the robot dog lively and cute.
4. **Trot smoothly accompanies gait and hardware optimization:**
   Adopt Trot diagonal gait movement algorithm. We used Jacobian transpose matrix analysis to raise the stance to reduce calf torque, and attached rubber wheels to the soles of the feet, which successfully solved the initial pain points of "soft feet" and "slippage" and ensured that the robot dog could follow safely on the floor at home.

The following are the detailed remote control functions of the current implementation:

System status and security control:

- **Wake Up:**
  As a command to wake up and unlock the robot dog. After clicking, the robot dog will slowly rise from the prone state to the standard standing posture at a smooth linear speed, and release the safety lock of other movements, ready for interaction.
- **Lying down to store (Sleep):**
  Let the robot dog fold its limbs smoothly and return to a resting position close to the ground. This function can reduce the standby power consumption of the motor, and also allows users to safely place it in a corner or on the desktop when not in operation.

Realistic interaction and gesture performance:

Action name	perform action
wave left hand	When this function is triggered, the robot dog will automatically fine-tune the stance of the remaining three legs to stabilize its center of gravity, and then lift its left front foot to swing back and forth, simulating a pet's realistic movements of actively greeting or asking for a pet.
wave right hand	Logically symmetrical to waving the left hand, the robot dog will shift its center of gravity to the left and lift its right front foot to swing, providing more diverse interactive responses for home companionship.
Keep your head high	The front legs are extended and the hind legs are slightly retracted, so that the front half of the body is raised, accurately simulating the situation of a puppy looking up at its owner, or the front half of the body stretching.
Raise your butt	The front legs are shortened and the hind legs are lengthened, making the body appear lower in front and higher in back. This function restores the appearance of a real dog when inviting its owner to play games and preparing to pounce.
rocking back and forth	This creates a continuous and smooth forward and backward displacement of the fuselage's center of gravity. Through this cyclic action, the robot dog can show the dynamic vitality similar to that of a pet when it is expecting, excited and acting coquettishly.
go forward	Long press this button, the robot dog will move continuously forward in Trot (diagonal gait). With the anti-slip design on the soles of the feet, it can smoothly follow the user on the home floor.
in the future	Perform a backward movement. In order to overcome the phenomenon that the multi-link mechanism easily loses balance when reversing, this function specifically incorporates anti-mopping leg lift compensation internally to ensure that the robot dog can still maintain a smooth and stable pace when moving away or retreating.

2.3 Functions that can be added in the future

The recent technological trend of companion robots has strongly focused on emotion perception and two-way interaction. In order to enhance the immersive experience of digital pets, the following functions are expected to be implemented in the future:

1. **Dynamic bionic dog head and visual lens:**
   It is expected that a set of physical dog heads with independent degrees of freedom will be added to the front of the fuselage, and a micro camera (such as an ESP32-CAM module) and a distance sensor will be built into the head to give the robot dog environmental vision and object recognition capabilities.
2. **Tactile and gesture interactive feedback:**
   Combining vision and sensors, when the robot dog detects the approach of human hands, it can trigger exclusive autonomous interactive behaviors. For example: when the palm of the hand stays in front, the robot dog will actively lower its neck to make a coquettish action of "leaning its head on the hands"; or when it senses the owner's wave, the robot dog can make high-frequency small steps and rock the body back and forth, making a dynamic response of "excited anticipation", greatly improving the emotional resonance when interacting with the elderly or children living alone.

3. **Integrating generative AI dialogue and companionship:**
   With the popularization of AI large language models, in the future, the API can be connected through Wi-Fi and equipped with a microphone/speaker module, so that the robot dog can not only understand the owner's voice commands, but also provide real-time feedback actions and vocal companionship in response to the owner's emotional changes.

3. Description of the overall electromechanical architecture and development of the finished product

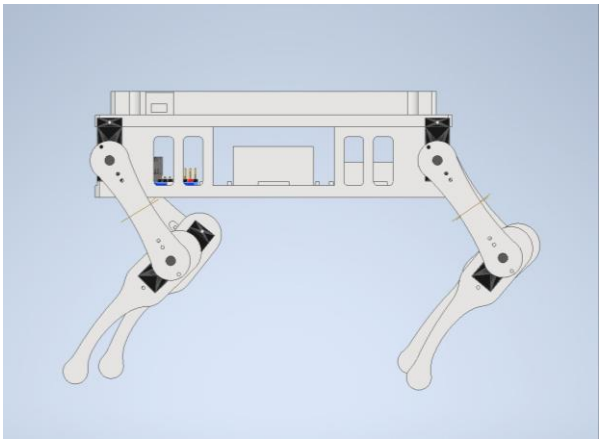
3.1 Overall system architecture

The finished product of this topic is a small 8-degree-of-freedom quadruped robot dog. The system integrates the mechanical structure, servo motor, ESP32 main controller, PCA9685 servo control board, UBEC 8A power supply module and DC-DC Mini560 step-down module. The overall design goal is to build a small quadruped robot platform that can be controlled wirelessly and perform basic movement and interaction actions.

3.2 Mechanical structure and mechanism adjustment

In terms of mechanical architecture, this work adopts a four-legged eight-joint configuration. Each foot is driven by two servo motors, corresponding to the thigh and calf joints respectively. This

architecture is simpler than a three-degree-of-freedom quadruped robot. Although it sacrifices lateral movement capabilities, it reduces the difficulty of mechanism design and control. It can still complete basic actions such as standing, moving forward, retreating, and changing postures. Each foot can be regarded as a two-link mechanism. By adjusting the angle of the servo motors of the thigh and calf, the foot can be moved to different positions to form a gait or interactive posture.



When the initial assembly test was completed, although the robot dog had already stepped, it was prone to standing still and failing to move forward effectively due to insufficient friction on its soles. Therefore, rubber wheels were attached to the soles of the feet to increase friction, and walking performance was significantly improved. Since the overall center of gravity was originally tilted back and the front feet were still slipping, the battery position was moved from the center of the body to the front to increase the positive force of the front feet and the friction on the ground, improving actual walking stability.

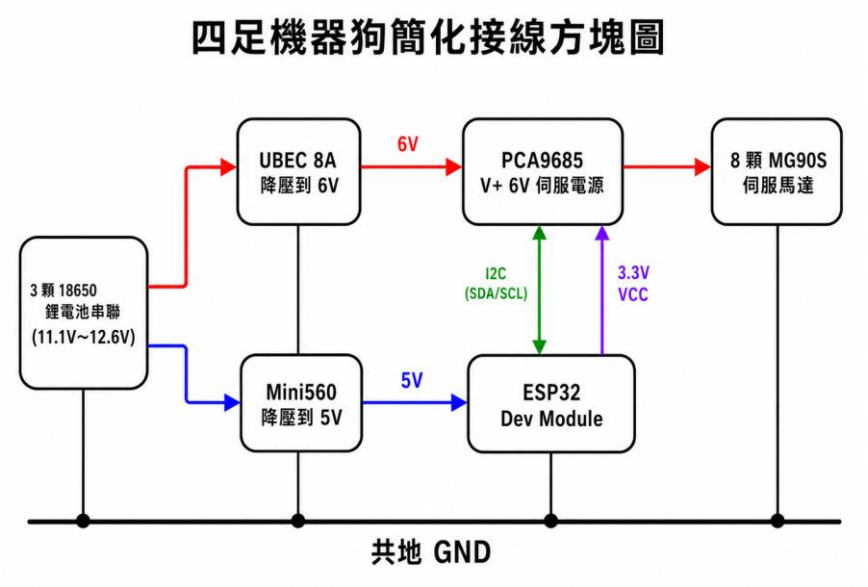
3.3 Electronic control architecture

In terms of electronic architecture, ESP32 serves as the main control core, responsible for receiving user operation instructions and controlling the overall action state. Since this work needs to control 8 servo motors at the same time, PCA9685 is used as the PWM signal expansion module. ESP32 and PCA9685 communicate through I2C, and PCA9685 then outputs multiple PWM signals to each servo motor. This design can reduce the burden of ESP32 directly controlling multiple servo motors and improve the stability of servo control signals.

3.4 Power supply architecture and stability design

In terms of power supply design, since multiple servo motors will generate large instantaneous currents when operating at the same time, this system uses UBEC 8A to provide the 6V power supply required by the servo motors to increase the stability of the motor output. The control circuit part uses the DC-DC Mini560 step-down module to provide the required voltage, so that the motor power supply and the control circuit power supply can be planned separately. This design can reduce the impact of voltage fluctuations on ESP32 caused by servo motor startup or load changes, preventing the main controller from restarting or Wi-Fi control interruption. Although the motor power supply

and the control circuit power supply are separated, each module still needs to share the ground so that the PWM control signal has a common reference potential.



3.5 Summary

Overall, the development focus of this work is to integrate the four-legged linkage mechanism, multi-servo motor control, stable power supply and wireless operating interface into a practical platform. Through mechanism adjustment, improvement of sole friction and battery position configuration, this system can complete basic walking and interactive display under limited hardware specifications, while retaining expansion space for subsequent addition of sensing modules.

4. List of MCU chip modules and peripheral chip modules used

The MCU used in this topic is ESP32 as the main control core, and is equipped with the PCA9685 servo motor drive module to control the 8 MG90S servo motors of the quadruped robot dog. The power supply uses a ubec transformer.

The following are the total modules used:

1. **Main control chip module: ESP32 Dev Module**
2. **Servo motor drive module: PCA9685**
3. **Motor: MG90S micro servo motor**
4. **Communication interface chip: CP2102**
5. **Power supply and protection hardware: 18650 lithium batteries connected in series and ubec transformer**

4.1 Main control chip module: ESP32 Dev Module

ESP32 is a high-performance microcontroller with a dual-core 32-bit processor and a main frequency of up to 240MHz. It has built-in 2.4 GHz Wi-Fi and dual-mode Bluetooth functions, combining powerful computing capabilities and IoT communication advantages.
The main reasons for choosing ESP32 in this topic are as follows:

1. Inverse kinematics calculation: This topic requires the calculation of the inverse kinematics trajectory of the robot dog and the planning of the walking gait.
2. Wireless remote control: After establishing a hotspot using the built-in Wi-Fi, and setting up a WebServer internally, we can use the mobile browser to directly send commands to the robot dog.
3. System control: Send control instructions to PCA9685 through I2C (SCL and SCA) to coordinate the actions of the entire machine

4.2 Servo motor drive module: PCA9685

PCA9685 is a 16-channel PWM controller based on I2C bus communication, with 12-bit output resolution and an independent motor power input terminal block.
The main reasons for choosing PCA9685 in this topic are as follows:

1. Expanded control channels: ESP32 cannot provide more than 8 absolutely stable hardware PWM signals at the same time. PCA9685 perfectly solves the problem of insufficient pin positions and is responsible for generating 50Hz control signals and driving 8 joint motors at the same time.
2. Reduce the burden on the main control: With its built-in clock generator, the ESP32 only needs to send the I2C signal of the target angle once, and the PCA9685 will continue to output the PWM wave of that angle, greatly freeing up the CPU resources of the ESP32 to process more important gait mathematics.
3. Power shunting: The module separates the logic junction (ESP32) and the motor terminal block independently in the circuit design, which can realize the physical shunting of the power supply and prevent the noise from the motor end from being directly poured into the digital communication line.

4.3 Motor: MG90S

MG90S is a micro servo motor with an output torque of approximately 2.2 kg-cm (at 6V). This topic uses rotation specifications of 0-180 degrees. And this motor has a metal gear structure, which greatly improves durability and impact resistance.
The main reasons for choosing MG90S in this topic are as follows:

1. Joint dynamics: The entire machine uses a total of 8 MG90S, which serve as the "hip joint (thigh)" and "knee joint (calf)" of the four legs to execute precise angles calculated by inverse kinematics.
2. Withstand dynamic impact: When the robot dog performs actions such as walking, lifting its butt, or sitting down and waving, the joints need to bear the weight of the body and dynamic acceleration. The characteristics of its metal gear effectively prevent tooth chipping and other

damage problems that occur when the center of gravity shifts frequently or when the rotor is slightly blocked.

4.4 Communication interface chip: CP2102

CP2102 is a USB to UART (serial port) bridge chip that converts the computer's USB signal into a signal that can be read by a microcontroller.
In this topic, since the ESP32 you purchased already contains this chip, it is described separately from the ESP32 because of its importance:

1. Program burning: As a communication bridge between the computer and ESP32, it is responsible for writing the compiled C++ program code into the chip's memory at a stable speed.
2. Real-time system monitoring: During the development and calibration stage of the robot dog, key system data such as motor angle, IP address, Wi-Fi status, etc. are sent back to the computer screen in real time through the 115200 baud rate, which is an extremely important debugging and monitoring channel.

4.5 Series 18650 Lithium Battery with UBEC Transformer

Three high-discharge rate 18650 lithium-ion batteries are connected in series (providing a high voltage of about 12.0V ~ 12.6V), and combined with a UBEC (the maximum allowable current of this UBEC is 8A) to efficiently convert the high voltage into an extremely stable 6V power supply with high current output capability.
The main reasons for choosing 18650 lithium battery and UBEC in this topic are as follows:

1. Overcome high current: When 8 MG90S motors support the dog's body or swing violently at the same time, the instantaneous current can reach more than 5A, and the maximum is about 7V after calculation. The general power bank or linear buck will collapse instantly, but the 18650 lithium battery and UBEC can provide enough amperage to ensure that the motor is not soft.
2. Stabilize the system voltage: When experiencing high-load operations, UBEC can pin the voltage to 5V, preventing the voltage from being pulled down by the motor and triggering the restart of the ESP32. This is the key to completely eliminating unknown jitter and crashes of the robot dog.

4.6 Summary

The following table is a list of MCU chip modules and peripheral chip modules:

Mod category	Chip/module name	Main specifications and features	Core functions in the system
MCU chip module	ESP32 Dev Module	(1)Dual-core 32-bit processor (2)2.4 GHz Wi-Fi module (3)Supports I2C and hardware timer interrupts	(1) Responsible for the inverse kinematics calculation and gait planning of the entire machine (2) Establish a Wi-Fi hotspot and run WebServer to receive remote control commands from the web page. (3) Control the 16-channel PWM driver chip through the I2C bus.
Servo motor drive module	PCA9685 servo motor driver board	(1)16-channel independent PWM output (2)12-bit output resolution (3)I2C communication interface (4) Equipped with independent motor power supply terminal block	(1) Expand the PWM pin of the MCU to exclusively generate 50Hz servo motor control signals. (2) Isolate the high current of the motor to prevent the motor from interfering with the ESP32 control core when pumping load.
Servo motor	MG90S micro servo motor	(1) Metal gear, working voltage 4.8V~6.0V (2)Rotation angle 0~180 degrees	(1) The joint structure that serves as a quadrupedal robot dog (4 legs × 2 core segments). (2) Withstand dynamic impacts and reduce damage problems.
Communication interface chip	CP2102 (integrated on ESP32)	USB to serial port bridge chip	(1) Responsible for program burning between the microcontroller and the computer.
power supply	18650 lithium battery	(1) Multiple 18650 lithium batteries connected in series for power supply (2) Provide voltage of about 11.1V~12.6V (3)Have high discharge capacity	(1) As the main energy source of the overall system (2) Provide power required for servo motors and control circuits (3) Use a transformer to supply stable voltage to improve the power supply stability of the robot dog

			while walking.
Transformer	UBEC 8A transformer module / Mini560 step-down module	(1)UBEC can reduce the voltage to about 6V for use by the motor (2)Mini560 can reduce the voltage to about 5V for use by ESP32 (3) Adopt separate power supply and common ground design for power and control circuits	(1) Provide stable high-current power supply for MG90S servo motor (2) Reduce voltage fluctuations caused by instantaneous load pumping of the motor (3) Avoid ESP32 restarting or Wi-Fi interruption due to voltage instability

5. Structure and description of the developed MCU program

The MCU program of this topic uses ESP32 as the main control core, and uses the PCA9685 servo motor drive module to control the 8 MG90S servo motors of the quadruped robot dog. The program is mainly responsible for receiving operating instructions from the mobile web page and controlling the robot dog to complete actions such as standing, lying down, moving forward, retreating, waving, raising its head, raising its buttocks, and rocking back and forth according to the instructions.

The overall program architecture can be divided into five parts:

1. Wi-Fi and web server: building a mobile control interface
2. Servo motor control: convert angle into PWM signal
3. Robot dog status control: determine the current action to be performed
4. Gait and inverse kinematics: used to calculate the movement trajectories of the four feet
5. Special action functions, such as waving, raising head, and rocking back and forth.

5.1 Program initialization and hardware architecture

The program will initially load the libraries required for ESP32 Wi-Fi, web server, I2C and PCA9685 servo motor drivers:

```
#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>
#include <math.h>
```

Among them, ESP32 will create a Wi-Fi hotspot named Quadruped_Dog. After the mobile phone is connected, you can search the default IP location through the browser to enter the control page. PCA9685 communicates with ESP32 through I2C and is responsible for outputting PWM signals to control 8 servo motors.

```
const char* ssid = "Quadruped_Dog";
const char* password = "password123";
WebServer server(80);
```

Since the quadruped robot dog has a total of 8 servo motors, if all of them are directly outputted by ESP32 to PWM, it will cause insufficient pin positions and unstable control current. Therefore, PCA9685 is used as the PWM expansion module. ESP32 only needs to transmit data through I2C to control multiple servo motors.

5.2 Mechanism parameters and motor calibration

We will first set the actual mechanical dimensions of the robot dog in the program, including thigh length, calf length and standing height:

```
const float L1 = 0.06;      // 大腿長度 (m)
const float L2 = 0.08;      // 小腿長度 (m)
const float H = 0.12;       // 機身預設高度 (m)
```

These parameters will be used in subsequent inverse kinematics calculations, allowing the program to calculate the angle at which the thigh and calf motors should rotate based on the toe target position. In addition, the program also sets the stride length and leg lift height of the left and right feet:

```
const float S_L = 0.047;
const float S_R = 0.049;

const float h_step_front = 0.007;
const float h_step_back  = 0.007;

const float T_cycle = 0.45;
```

Where S_L and S_R represent the strides on the left and right sides respectively. Since the torque of the left and right motors of the actual mechanism, the friction of the soles of the feet, and the distribution of the center of gravity are not exactly the same, the left and right strides are slightly different to correct the offset problem during actual walking. In addition, we also specially lowered the span of the robot dog when walking and increased the leg swing frequency to reduce the air time of the robot dog's legs when changing feet and avoid instability during the movement.

The motor angles of the robot dog are not all 90 degrees as the standing posture. Instead, the standing and lying posture angles are obtained after correction through actual tests. And when deciding the initial stance, we also take the motor torque into consideration to avoid soft foot problems caused by excessive torque on the calf motor during walking:

```
// 一上電時的角度 (趴下)
const int restThigh[4] = {135, 56, 163, 35};
const int restCalf[4]  = {162, 61, 176, 39};

// 站立基準角度
const int baseThigh[4] = {119, 87, 108, 92};
const int baseCalf[4]  = {83, 136, 69, 155};
```

The four array elements represent the left front foot, the right front foot, the left rear foot and the right rear foot in order. Through these correction angles, the program can apply the theoretical kinematics calculation results to the actual mechanism.

5.3 Mobile web control interface

This topic uses ESP32 to create a web control interface as shown in the figure below:

The webpage contains multiple buttons, and users can directly operate the robot dog through their mobile phones. The control functions include activating standing posture, lying down, waving left hand, waving right hand, raising head, raising buttocks, rocking back and forth, forward and

backward. The button in the web page will call the sendCmd() function to send different status values to ESP32:

```
<div class='card'>
  <button class="btn btn-stand" onclick="sendCmd(2)"> 啟動站姿 (Wake Up)</button>
  <button class="btn btn-sleep" onclick="sendCmd(3)"> 趴下收納 (Sleep)</button>

  <div class="flex-container">
    <button class="btn btn-wave" onclick="sendCmd(4)"> 揮左手</button>
    <button class="btn btn-wave-right" onclick="sendCmd(5)"> 揮右手</button>
  </div>

  <div class="flex-container">
    <button class="btn btn-pose" onclick="sendCmd(6)"> 抬高頭</button>
    <button class="btn btn-pose" onclick="sendCmd(7)"> 抬高屁股</button>
  </div>
  <button class="btn btn-sway" onclick="sendCmd(8)"> 前後搖擺 (Sway)</button>

  <hr>
  <button class="btn" onmousedown="sendCmd(1)" onmouseup="sendCmd(0)" onmouseleave="sendCmd(0)" ontouchstart="sendCmd(1); event.preventDefault(
  <button class="btn btn-stop" onmousedown="sendCmd(-1)" onmouseup="sendCmd(0)" onmouseleave="sendCmd(0)" ontouchstart="sendCmd(-1); event.preve
</div>
```

The instructions sent from the mobile phone will be sent to ESP32 in the form of http request:

```
function sendCmd(state) {
  if(lastState === state && state !== 2 && state !== 3 && state !== 4 && state !== 5 && state !== 6 && stat
  lastState = state;
  fetch('/cmd?state=' + state);
}
```

After ESP32 receives the command, it will switch different actions according to the value of state.

5.4 State machine control logic

The program uses robot_state as the main state variable, and uses is_awake to determine whether the robot dog has entered the standing state from the lying state. Different status values represent different functions:

incoming_state value	function
1	move forward
-1	Back
0	Stop and return to standing position
2	Start stance
3	Lie down to store
4	wave left hand
5	wave right hand
6	Keep your head high
7	Raise your butt

8	rocking back and forth

```
if (server.hasArg("state")) {
  int incoming_state = server.arg("state").toInt();

  if (incoming_state == 2) {
    if (!is_awake) { wakeUpSequence(); is_awake = true; }
    robot_state = 0;
  }
  else if (incoming_state == 3) {
    if (is_awake) { sleepSequence(); is_awake = false; }
    robot_state = 0;
  }
  else if (incoming_state == 4) {
    if (is_awake) { robot_state = 0; waveLeftHand(); }
  }
  else if (incoming_state == 5) {
    if (is_awake) { robot_state = 0; waveRightHand(); }
  }
  else if (incoming_state == 6) {
    if (is_awake) { raiseHeadPose(); robot_state = 6; }
  }
  else if (incoming_state == 7) {
    if (is_awake) { raiseButtPose(); robot_state = 7; }
  }
  else if (incoming_state == 8) {
    if (is_awake) { swayFrontBack(); robot_state = 0; }
  }
  else if (is_awake) {
    robot_state = incoming_state;
  }
}
```

This design prevents the robot dog from performing walking or special actions before it stands up, and avoids the risk of excessive momentary force on the motor and damage to the mechanism.

5.5 Main loop and gait update

In loop(), ESP32 will continue to receive control instructions from the mobile phone and update its actions every 20 ms. This refresh rate is used because 20 ms is approximately equal to 50 Hz, which is similar to the PWM update frequency commonly used in servo motors.

```
void loop() {
  server.handleClient();

  if (!is_awake) return;

  static unsigned long lastUpdate = 0;
  unsigned long now = millis();

  if (now - lastUpdate >= 20) {
    lastUpdate = now;
    if (robot_state == 1 || robot_state == -1) {
      current_time += 0.02;
      if (current_time >= T_cycle) current_time -= T_cycle;

      float phi_FR = fmod(current_time / T_cycle, 1.0);
      float phi_HL = fmod(current_time / T_cycle, 1.0);
      float phi_FL = fmod((current_time / T_cycle) + 0.5, 1.0);
      float phi_HR = fmod((current_time / T_cycle) + 0.5, 1.0);

      updateLeg(0,  1,  0, phi_FL, robot_state);
      updateLeg(4,  5,  1, phi_FR, robot_state);
      updateLeg(8,  9,  2, phi_HL, robot_state);
      updateLeg(12, 13, 3, phi_HR, robot_state);
    }
    else if (robot_state == 6 || robot_state == 7) {
    }
    else {
      current_time = 0;
      standStill();
    }
  }
}
```

When walking we use the Trot diagonal gait. The right front foot is synchronized with the left rear foot, the left front foot is synchronized with the right rear foot, and the phases of the two sets of feet differ by half a cycle. This allows the robot dog to complete basic walking movements even though it cannot achieve turning and lateral movement with only a 2-DOF leg structure.

5.6 Toe trajectory generation

When we design the gait of the robot dog, we do not directly specify the motor angle (forward kinematics). Instead, we first calculate the toe target position and then convert it into the motor angle by inverse kinematics. The toe trajectory is generated with the help of the getTrajectory() function.

```
void getTrajectory(float phi, float dir, float &x, float &y, int legIndex) {
  float current_h_step = (legIndex == 2 || legIndex == 3) ? h_step_back : h_step_front;
  float current_S = (legIndex == 0 || legIndex == 2) ? S_L : S_R;
  float x_offset = 0;
  float actual_dir = -dir;

  if (dir < 0) {
    current_S *= 0.7;
    if (legIndex == 0 || legIndex == 2) {
      current_S *= 0.82;
    }
    if (legIndex == 1) {
      current_S *= 1.20;
    }
    if (legIndex == 3) {
      current_S *= 1.02;
      current_h_step += 0.001;
    }
    if (legIndex == 0) {
      current_h_step += 0.010;
      x_offset = actual_dir * 0.006;
    }
    else if (legIndex == 1) {
      x_offset = actual_dir * 0.012;
    }
  }

  if (phi < 0.5) {
    float phi_norm = phi * 2.0;
    x = actual_dir * (-current_S / 2.0 + current_S * phi_norm) + x_offset;
    y = -H + current_h_step * sin(PI * phi_norm);
  }
  else {
    float phi_norm = (phi - 0.5) * 2.0;
    x = actual_dir * (current_S / 2.0 - current_S * phi_norm) + x_offset;
    y = -H;
  }
}
```

When phi < 0.5, the foot is in a swinging state, and the toes will be lifted and stepped; when phi >= 0.5, the feet will be in a supported state, and the toes remain at the height of the ground and move in the opposite direction to push the body. The program adds additional compensation for different feet when retreating. For example, when the left front foot steps back, the height of the leg lift is increased to avoid dragging the ground, while the right front foot increases the stride length to correct the problem of deviation when backing up. These compensations are adjusted by observing actual test results.

5.7 Inverse kinematics calculation

After the toe trajectory is generated, the program will call solveIK() to convert the toe coordinates into the angle of the thigh and calf.

```
void solveIK(float x, float y, float &theta1_deg, float &theta2_deg) {
  float D2 = x * x + y * y;
  float D = sqrt(D2);

  if (D > (L1 + L2)) {
    x = x * ((L1 + L2) * 0.99 / D);
    y = y * ((L1 + L2) * 0.99 / D);
    D2 = x * x + y * y;
    D = sqrt(D2);
  }

  float cos_theta2 = (D2 - L1 * L1 - L2 * L2) / (2.0 * L1 * L2);
  cos_theta2 = constrain(cos_theta2, -1.0, 1.0);
  float alpha = atan2(y, x);
  float cos_beta = (D2 + L1 * L1 - L2 * L2) / (2.0 * L1 * D);
  cos_beta = constrain(cos_beta, -1.0, 1.0);
  float beta = acos(cos_beta);

  theta1_deg = (alpha - beta) * 180.0 / PI;
  theta2_deg = acos(cos_theta2) * 180.0 / PI;
}
```

This function uses the thigh length L1 and calf length L2 and uses the kinematics of MATLAB simulation to calculate the joint angle of the 2-DOF leg. If the toe target exceeds the leg reachable range, the program will automatically retract the target point within the maximum leg length range to avoid motor angle calculation errors.

5.8 Motor angle output

After inverse kinematics calculates the theoretical angle, the program will add the actual corrected stance angle through updateLeg() and then output it to the corresponding motor.

```
void updateLeg(int thighID, int calfID, int legIndex, float phi, int walkDir) {
  float x, y, t1, t2;
  getTrajectory(phi, (float)walkDir, x, y, legIndex);
  solveIK(x, y, t1, t2);

  float thighDelta = (t1 - neutral_t1) * dirThigh[legIndex];
  float calfDelta  = (t2 - neutral_t2) * dirCalf[legIndex];

  int angleThigh = baseThigh[legIndex] + (int)thighDelta;
  int angleCalf  = baseCalf[legIndex]  + (int)calfDelta;

  updateServoLogic(thighID, angleThigh);
  updateServoLogic(calfID,  angleCalf);
}
```

Among them, dirThigh[] and dirCalf[] are used to correct the angle reversal problem caused by the different installation directions of the left and right motors.
The bottom-level motor output function is as follows:

```
void updateServoLogic(int id, int angle) {
  if (id < 0 || id == 2 || id == 3 || id == 6 || id == 7 || id == 10 || id == 11 || id > 13) return;
  angle = constrain(angle, 0, 180);
  int pulse = map(angle, 0, 180, SERVOMIN, SERVOMAX);
  pwm.setPWM(id, 0, pulse);
}
```

This function will first limit the motor angle to 0 to 180 degrees, and then convert the angle into the PWM pulse wave required by PCA9685. In addition, the program also limits that only the channels actually connected to the motor can output signals to avoid mistaken control of unused channels.

5.9 Stand up and lie down control

After the robot dog is powered on, it will first enter the lying down storage posture. After pressing "Start Standing Posture", it will execute wakeUpSequence() to stand up smoothly. This function uses linear interpolation to gradually move the motor from the lying angle to the standing angle.

```
float currentThigh = restThigh[j] + (baseThigh[j] - restThigh[j]) * progress;
float currentCalf  = restCalf[j]  + (baseCalf[j]  - restCalf[j])  * progress;
```

When lying down, use sleepSequence() to smoothly move the motor from the standing posture back to the retracted posture. This can prevent the motor from rotating rapidly in an instant and also reduce the impact of the mechanism.

5.10 Special action function

In addition to moving forward and backward, this program also adds a variety of display movements, including waving the left hand, waving the right hand, raising the head, raising the buttocks, and rocking back and forth. Taking the left hand as an example for waving, the program will first adjust the stance of the other three feet, shift the center of gravity to the supporting foot, then lift the left front foot and swing it back and forth:

```
void waveLeftHand() {
  const int FL_THIGH = 0; const int FL_CALF = 1; const int FL_INDEX = 0;
  int waveThigh[4] = {116, 96, 111, 84};
  int waveCalf[4]  = {92,  148, 78,  144};

  int shiftSteps = 30; int shiftDelay = 12;
  for (int i = 1; i <= shiftSteps; i++) {
    float p = (float)i / shiftSteps;
    for (int leg = 0; leg < 4; leg++) {
      int thighAngle = baseThigh[leg] + (int)((waveThigh[leg] - baseThigh[leg]) * p);
      int calfAngle  = baseCalf[leg]  + (int)((waveCalf[leg]  - baseCalf[leg])  * p);
      updateServoLogic(leg * 4, thighAngle); updateServoLogic(leg * 4 + 1, calfAngle);
    }
    delay(shiftDelay);
  }
  delay(400);

  float x_start = 0.0; float y_start = -H;
  float x_lift = 0.0;  float y_lift = -H + 0.030;
  float x_wave_front = 0.022; float x_wave_back = -0.022; float y_wave = -H + 0.035;
  int steps = 18; int delayTime = 18;

  auto holdWaveStance = [&]() {
    updateServoLogic(4, waveThigh[1]);  updateServoLogic(5, waveCalf[1]);
    updateServoLogic(8, waveThigh[2]);  updateServoLogic(9, waveCalf[2]);
    updateServoLogic(12, waveThigh[3]); updateServoLogic(13, waveCalf[3]);
  };

  for (int i = 0; i <= steps; i++) {
    float p = (float)i / steps;
    updateLegByXY(FL_THIGH, FL_CALF, FL_INDEX, x_start + (x_lift - x_start) * p, y_start + (y_lift - y_start) * p);
    holdWaveStance(); delay(delayTime);
  }
  delay(120);

  for (int wave = 0; wave < 3; wave++) {
    for (int i = 0; i <= steps; i++) {
      updateLegByXY(FL_THIGH, FL_CALF, FL_INDEX, x_wave_back + (x_wave_front - x_wave_back) * (float)i/steps, y_wave);
      holdWaveStance(); delay(delayTime);
    }
    for (int i = 0; i <= steps; i++) {
      updateLegByXY(FL_THIGH, FL_CALF, FL_INDEX, x_wave_front + (x_wave_back - x_wave_front) * (float)i/steps, y_wave);
      holdWaveStance(); delay(delayTime);
    }
  }

  for (int i = 0; i <= steps; i++) {
    float p = (float)i / steps;
    updateLegByXY(FL_THIGH, FL_CALF, FL_INDEX, x_lift + (x_start - x_lift) * p, y_lift + (y_start - y_lift) * p);
    holdWaveStance(); delay(delayTime);
  }
  delay(150);

  for (int i = 1; i <= shiftSteps; i++) {
    float p = (float)i / shiftSteps;
    for (int leg = 0; leg < 4; leg++) {
      int thighAngle = waveThigh[leg] + (int)((baseThigh[leg] - waveThigh[leg]) * p);
      int calfAngle  = waveCalf[leg]  + (int)((baseCalf[leg]  - waveCalf[leg])  * p);
      updateServoLogic(leg * 4, thighAngle); updateServoLogic(leg * 4 + 1, calfAngle);
    }
    delay(shiftDelay);
  }
  robot_state = 0;
}
```

Lifting the head and buttocks is accomplished by changing the height of the toes of the front and rear feet. For example, when raising the head, the front legs are extended and the hind legs are shortened, raising the front of the body; raising the buttocks does the opposite.

```
void raiseHeadPose() {
  int steps = 40; int delayTime = 15;
  Serial.println("執行 IK 平滑抬頭...");
  for (int i = 1; i <= steps; i++) {
    float p = (float)i / steps;
    float y_front = -H - (0.012 * p);
    float y_back  = -H + (0.015 * p);
    updateLegByXY(0,  1,  0, 0.0, y_front);
    updateLegByXY(4,  5,  1, 0.0, y_front);
    updateLegByXY(8,  9,  2, 0.0, y_back);
    updateLegByXY(12, 13, 3, 0.0, y_back);
    delay(delayTime);
  }
}
```

Rocking back and forth uses the sine function to produce smooth displacement:

```
void swayFrontBack() {
  const int FL_THIGH = 0;  const int FL_CALF = 1;  const int FL_INDEX = 0;
  const int FR_THIGH = 4;  const int FR_CALF = 5;  const int FR_INDEX = 1;
  const int HL_THIGH = 8;  const int HL_CALF = 9;  const int HL_INDEX = 2;
  const int HR_THIGH = 12; const int HR_CALF = 13; const int HR_INDEX = 3;

  float swayAmplitude = 0.025; // 調整為更安全的 2.5 公分大幅度擺動
  int cycles = 3;
  int stepsPerCycle = 80;
  int delayTime = 12;

  Serial.println("開始執行前後平滑搖擺...");

  for (int c = 0; c < cycles; c++) {
    for (int i = 0; i < stepsPerCycle; i++) {
      float angle = (float)i / stepsPerCycle * 2.0 * M_PI;
      float x_offset = swayAmplitude * sin(angle);

      updateLegByXY(FL_THIGH, FL_CALF, FL_INDEX, x_offset, -H);
      updateLegByXY(FR_THIGH, FR_CALF, FR_INDEX, x_offset, -H);
      updateLegByXY(HL_THIGH, HL_CALF, HL_INDEX, x_offset, -H);
      updateLegByXY(HR_THIGH, HR_CALF, HR_INDEX, x_offset, -H);

      delay(delayTime);
    }
  }
  standStill();
  current_time = 0;
  robot_state = 0;
}
```

These special movements all use the same inverse kinematics basis, only changing the target position of the toes, so the program structure has good scalability.

5.11 Summary

The MCU program of this topic uses ESP32 as the control core, receives user instructions through the mobile web page, and then uses PCA9685 to control 8 servo motors. The walking part adopts the Trot diagonal gait, and calculates the angles of each joint through the toe trajectory and 2-DOF inverse kinematics. The program also uses the standing posture and lying angle obtained from actual tests to enable theoretical kinematics to be applied to physical mechanisms. Overall, this program not only controls the motor rotation, but integrates the robot dog's mechanism size, gait planning, inverse kinematics, motor calibration and mobile phone control interface, so that the robot dog can complete basic walking and various display actions. If you want to add cameras, sensors or automatic navigation functions in the future, you can continue to expand on the current program structure.

6. General conclusion

This topic successfully designed and implemented a small four-legged robot dog with ESP32 as the core controller. The overall system integrates the 3D printing mechanism, MG90S servo motor, PCA9685 PWM drive module, 18650 battery power supply system, UBEC transformer, and mobile web control interface, allowing the robot dog to perform a variety of actions wirelessly. The work can not only complete basic standing, lying down, forward and backward, but also perform interactive displays such as waving, raising the head, raising the butt and rocking back and forth, showing the prototype of a domestic companion robot dog.

In terms of mechanism design, this work adopts a simplified design of four legs and eight joints, with each leg controlled by two servo motors. Although this architecture cannot perform lateral movement or complex posture control like high-end quadruped robots, it can effectively reduce production costs, reduce program control difficulty, and make the overall system more suitable for implementation within a limited time. Through actual testing, we found that the friction between the soles of the feet, the torque of the motor, and the distribution of the center of gravity of the body will greatly affect the walking stability. Therefore, we subsequently added rubber wheels, adjusted the battery position and gait parameters, and gradually stabilized the robot dog's mobile performance.

In terms of the electronic control system, the Wi-Fi function of ESP32 allows the robot dog to establish a dedicated hotspot, and users can operate it through a mobile browser, greatly improving the convenience of control. PCA9685 effectively solves the problems of insufficient PWM pins and unstable signals that may be encountered when ESP32 directly controls multiple servo motors. The power supply part adopts separate planning of power supply and control power supply and common ground, so that the motor will not easily cause ESP32 restart or Wi-Fi interruption when operating under high load, improving the overall system stability.

In terms of programming, this topic integrates state machine control, Trot diagonal gait, toe trajectory planning and 2-DOF inverse kinematics into the MCU program. Compared with directly specifying the motor angle, using the toe position with inverse kinematics is more flexible, and it is easier to adjust the stride length, leg lift height and compensation amount based on actual test results. In addition, through stance angle correction, the theoretical kinematic model can be more smoothly applied to the physical mechanism, improving movement deviations caused by assembly errors.

Overall, this topic has completed a low-cost four-legged robot dog platform with basic walking capabilities and interactive functions. During the implementation process, we not only learned about

microprocessor control, I2C communication, PWM motor drive and web remote control, but also deeply understood the impact of mechanical structure, power supply stability, friction and center of gravity distribution on the actual movement of the robot. In the future, if cameras, distance sensors, voice modules or AI dialogue functions can be further added, and the gait and body structure can be continuously optimized, this robot dog will be further developed from a basic remote control platform into a home smart robot with the capabilities of environmental perception and companion interaction.

7. All MCU programs

```
#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>
#include <math.h>

const char* ssid = "Quadruped_Dog";
const char* password = "password123";
WebServer server(80);

Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver();

#define SERVOMIN  110
#define SERVOMAX  500

const float L1 = 0.06; // Thigh length (m)
const float L2 = 0.08; // calf length (m)
const float H = 0.12; // Default height of the fuselage (m)

const float S_L = 0.045;
const float S_R = 0.049;

const float h_step_front = 0.007;
const float h_step_back  = 0.007;

const float T_cycle = 0.45;

const int restThigh[4] = {135, 56, 163, 35};
const int restCalf[4]  = {162, 61, 176, 39};

const int baseThigh[4] = {114, 87, 108, 92};
const int baseCalf[4]  = {88, 136, 69, 155};

//Leg number: 0=front left FL, 1=front right FR, 2=rear left HL, 3=rear right HR
const float dirThigh[4] = {1.0, -1.0, 1.0, -1.0};
```

```
const float dirCalf[4]  = {1.0, -1.0, 1.0, -1.0};

float neutral_t1 = 0;
float neutral_t2 = 0;
float current_time = 0;

int robot_state = 0;
bool is_awake = false;

const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta charset='UTF-8'>
  <meta name='viewport' content='width=device-width, initial-scale=1.0, user-scalable=no'>
<title>Four-legged robot dog remote control</title>
  <style>
    body { font-family: 'Segoe UI', sans-serif; text-align: center; background-color: #f4f4f9; padding:
20px; user-select: none; -webkit-user-select: none; }
    .card { background: white; border-radius: 10px; padding: 30px 15px; margin: 20px auto; box-
shadow: 0 4px 8px rgba(0,0,0,0.1); max-width: 400px; }
    .btn { display: block; width: 90%; padding: 35px 0; margin: 20px auto; font-size: 28px; font-
weight: bold; color: white; background-color: #007bff; border: none; border-radius: 15px; cursor:
pointer; box-shadow: 0 6px 10px rgba(0,0,0,0.2); transition: transform 0.1s; }
    .btn:active { transform: scale(0.95); }
    .btn-stand { background-color: #28a745; padding: 20px 0; font-size: 22px; }
    .btn-sleep { background-color: #6c757d; padding: 20px 0; font-size: 22px; margin-top: 10px; }
    .btn-wave { background-color: #9c27b0; padding: 20px 0; font-size: 22px; margin-top: 10px; }
    .btn-wave-right { background-color: #e91e63; padding: 20px 0; font-size: 22px; margin-top:
10px; }
    .btn-pose { background-color: #ff9800; padding: 20px 0; font-size: 22px; margin-top: 10px; }
    .btn-sit-wave { background-color: #ff5722; padding: 20px 0; font-size: 22px; margin-top: 10px; }
    .btn-sway { background-color: #00bcd4; padding: 20px 0; font-size: 22px; margin-top: 10px; }
    .btn-stop { background-color: #dc3545; }
    h2 { color: #333; }
    .flex-container { display: flex; justify-content: space-between; width: 90%; margin: 0 auto; }
    .flex-container .btn { width: 48%; margin: 10px 0; }
  </style>
</head>
<body>
<h2>Four-legged robot dog (< < configuration)</h2>
  <div class='card'>
<button class="btn btn-stand" onclick="sendCmd(2)"> 🔋 Wake Up</button>
<button class="btn btn-sleep" onclick="sendCmd(3)"> 💤 Lie down and store (Sleep)</button>

    <div class="flex-container">
<button class="btn btn-wave" onclick="sendCmd(4)"> 👋 wave left hand</button>
```

```
<button class="btn btn-wave-right" onclick="sendCmd(5)">👋 wave your right hand</button>
    </div>

    <div class="flex-container">
<button class="btn btn-pose" onclick="sendCmd(6)">🐶 Sit</button>
<button class="btn btn-pose" onclick="sendCmd(7)">🔽 Raise your butt</button>
    </div>

<button class="btn btn-sit-wave" onclick="sendCmd(9)">🐾 wave left hand (sitting position
only)</button>
<button class="btn btn-sway" onclick="sendCmd(8)">↕ Sway</button>

    <hr>
<button class="btn" onmousedown="sendCmd(1)" onmouseup="sendCmd(0)"
onmouseleave="sendCmd(0)" ontouchstart="sendCmd(1); event.preventDefault();"
ontouchend="sendCmd(0); event.preventDefault();">Forward</button>
<button class="btn btn-stop" onmousedown="sendCmd(-1)" onmouseup="sendCmd(0)"
onmouseleave="sendCmd(0)" ontouchstart="sendCmd(-1); event.preventDefault();"
ontouchend="sendCmd(0); event.preventDefault();">Backward</button>
  </div>
  <script>
    let lastState = -100;
    function sendCmd(state) {
      if(lastState === state && state !== 2 && state !== 3 && state !== 4 && state !== 5 && state !==
6 && state !== 7 && state !== 8 && state !== 9) return;
      lastState = state;
      fetch('/cmd?state=' + state);
    }
  </script>
</body>
</html>
)rawliteral";

void updateServoLogic(int id, int angle);
void solveIK(float x, float y, float &theta1_deg, float &theta2_deg);
void getTrajectory(float phi, float dir, float &x, float &y, int legIndex);
void updateLeg(int thighID, int calfID, int legIndex, float phi, int walkDir);
void updateLegByXY(int thighID, int calfID, int legIndex, float x, float y);
void standStill();
void restStill();
void wakeUpSequence();
void sleepSequence();
void waveLeftHand();
void waveRightHand();
void raiseHeadPose();
void raiseButtPose();
void swayFrontBack();
```

```
void sitWaveSequence();

void setup() {
  Serial.begin(115200);

  solveIK(0.0, -H, neutral_t1, neutral_t2);

  pwm.begin();
  pwm.setPWMFreq(50);
  delay(100);

  restStill();

  WiFi.softAP(ssid, password);
  Serial.println("WiFi Ready.");

  server.on("/", []() {
    server.send(200, "text/html", index_html);
  });

  server.on("/cmd", []() {
    if (server.hasArg("state")) {
      int incoming_state = server.arg("state").toInt();

      if (incoming_state == 2) {
        if (!is_awake) { wakeUpSequence(); is_awake = true; }
        robot_state = 0;
      }
      else if (incoming_state == 3) {
        if (is_awake) { sleepSequence(); is_awake = false; }
        robot_state = 0;
      }
      else if (incoming_state == 4) {
        if (is_awake) { robot_state = 0; waveLeftHand(); }
      }
      else if (incoming_state == 5) {
        if (is_awake) { robot_state = 0; waveRightHand(); }
      }
      else if (incoming_state == 6) {
        if (is_awake) {
          if(robot_state != 6) { raiseHeadPose(); }
          robot_state = 6;
        }
      }
      else if (incoming_state == 7) {
        if (is_awake) { raiseButtPose(); robot_state = 7; }
      }
```

```
    else if (incoming_state == 8) {
      if (is_awake) { swayFrontBack(); robot_state = 0; }
    }
    else if (incoming_state == 9) {
      if (is_awake) {
        if (robot_state != 6) {
          raiseHeadPose();
          delay(500);
        }
        sitWaveSequence();
        robot_state = 6;
      }
    }
    else if (is_awake) {
      robot_state = incoming_state;
    }
   }
   server.send(200, "text/plain", "OK");
  });

  server.begin();
}

void loop() {
  server.handleClient();

  if (!is_awake) return;

  static unsigned long lastUpdate = 0;
  unsigned long now = millis();

  if (now - lastUpdate >= 20) {
    lastUpdate = now;

    if (robot_state == 1 || robot_state == -1) {
      current_time += 0.02;
      if (current_time >= T_cycle) current_time -= T_cycle;

      float phase_shift = (robot_state == -1) ? 0.5 : 0.0;

      float phi_FR = fmod((current_time / T_cycle) + phase_shift, 1.0);
      float phi_HL = fmod((current_time / T_cycle) + phase_shift, 1.0);
      float phi_FL = fmod((current_time / T_cycle) + 0.5 + phase_shift, 1.0);
      float phi_HR = fmod((current_time / T_cycle) + 0.5 + phase_shift, 1.0);

      updateLeg(0,  1,  0, phi_FL, robot_state);
      updateLeg(4,  5,  1, phi_FR, robot_state);
```

```
      updateLeg(8,  9,  2, phi_HL, robot_state);
      updateLeg(12, 13, 3, phi_HR, robot_state);
    }
    else if (robot_state == 6 || robot_state == 7) {
// maintain static pose
    }
    else {
      current_time = 0;
      standStill();
    }
  }
}

void wakeUpSequence() {
  int steps = 40;
  int delayTime = 10;
  const int thighIDs[4] = {0, 4, 8, 12};
  const int calfIDs[4]  = {1, 5, 9, 13};

  for (int i = 1; i <= steps; i++) {
    float progress = (float)i / steps;
    for (int j = 0; j < 2; j++) {
      float currentThigh = restThigh[j] + (baseThigh[j] - restThigh[j]) * progress;
      float currentCalf  = restCalf[j]  + (baseCalf[j]  - restCalf[j])  * progress;
      updateServoLogic(thighIDs[j], (int)currentThigh);
      updateServoLogic(calfIDs[j],  (int)currentCalf);
    }
    delay(delayTime);
  }
  delay(150);
  for (int i = 1; i <= steps; i++) {
    float progress = (float)i / steps;
    for (int j = 2; j < 4; j++) {
      float currentThigh = restThigh[j] + (baseThigh[j] - restThigh[j]) * progress;
      float currentCalf  = restCalf[j]  + (baseCalf[j]  - restCalf[j])  * progress;
      updateServoLogic(thighIDs[j], (int)currentThigh);
      updateServoLogic(calfIDs[j],  (int)currentCalf);
    }
    delay(delayTime);
  }
}

void sleepSequence() {
  int steps = 40;
  int delayTime = 10;
  const int thighIDs[4] = {0, 4, 8, 12};
  const int calfIDs[4]  = {1, 5, 9, 13};
```

```
  for (int i = 1; i <= steps; i++) {
    float progress = (float)i / steps;
    for (int j = 0; j < 4; j++) {
      float currentThigh = baseThigh[j] + (restThigh[j] - baseThigh[j]) * progress;
      float currentCalf  = baseCalf[j]  + (restCalf[j]  - baseCalf[j])  * progress;
      updateServoLogic(thighIDs[j], (int)currentThigh);
      updateServoLogic(calfIDs[j],  (int)currentCalf);
    }
    delay(delayTime);
  }
}

void restStill() {
  for (int i = 0; i < 4; i++) {
    updateServoLogic(i * 4,     restThigh[i]);
    updateServoLogic(i * 4 + 1, restCalf[i]);
  }
}

void standStill() {
  for (int i = 0; i < 4; i++) {
    updateServoLogic(i * 4,     baseThigh[i]);
    updateServoLogic(i * 4 + 1, baseCalf[i]);
  }
}

void updateServoLogic(int id, int angle) {
  if (id < 0 || id == 2 || id == 3 || id == 6 || id == 7 || id == 10 || id == 11 || id > 13) return;
  angle = constrain(angle, 0, 180);
  int pulse = map(angle, 0, 180, SERVOMIN, SERVOMAX);
  pwm.setPWM(id, 0, pulse);
}

void waveLeftHand() {
  const int FL_THIGH = 0; const int FL_CALF = 1; const int FL_INDEX = 0;
  int shiftSteps = 40; int shiftDelay = 15;

  float x_shift = 0.025;
  float y_brace_FR = -H -0;
  float y_crouch_HL = -H + 0.010;
  float y_crouch_HR = -H + 0.016;

  for (int i = 1; i <= shiftSteps; i++) {
    float p = (float)i / shiftSteps;
    updateLegByXY(0,  1,  0, x_shift * p, -H);
    updateLegByXY(4,  5,  1, x_shift * p, -H + (-0.0 * p));
```

```
    updateLegByXY(8,  9,  2, x_shift * p, -H + (0.010 * p));
    updateLegByXY(12, 13, 3, x_shift * p, -H + (0.016 * p));
    delay(shiftDelay);
  }
  delay(400);

  float x_start = x_shift; float y_start = -H;
  float x_lift = x_shift + 0.020; float y_lift = -H + 0.040;
  float x_wave_front = x_lift + 0.020; float x_wave_back = x_lift - 0.020;
  float y_wave = y_lift;

  auto holdWaveStance = [&]() {
    updateLegByXY(4,  5,  1, x_shift, y_brace_FR);
    updateLegByXY(8,  9,  2, x_shift, y_crouch_HL);
    updateLegByXY(12, 13, 3, x_shift, y_crouch_HR);
  };

  int steps = 20; int delayTime = 15;

  for (int i = 0; i <= steps; i++) {
    float p = (float)i / steps;
    updateLegByXY(FL_THIGH, FL_CALF, FL_INDEX, x_start + (x_lift - x_start) * p, y_start +
(y_lift - y_start) * p);
    holdWaveStance(); delay(delayTime);
  }
  delay(120);

  for (int wave = 0; wave < 3; wave++) {
    for (int i = 0; i <= steps; i++) {
      updateLegByXY(FL_THIGH, FL_CALF, FL_INDEX, x_wave_back + (x_wave_front -
x_wave_back) * (float)i/steps, y_wave);
      holdWaveStance(); delay(delayTime);
    }
    for (int i = 0; i <= steps; i++) {
      updateLegByXY(FL_THIGH, FL_CALF, FL_INDEX, x_wave_front + (x_wave_back -
x_wave_front) * (float)i/steps, y_wave);
      holdWaveStance(); delay(delayTime);
    }
  }

  for (int i = 0; i <= steps; i++) {
    float p = (float)i / steps;
    updateLegByXY(FL_THIGH, FL_CALF, FL_INDEX, x_lift + (x_start - x_lift) * p, y_lift +
(y_start - y_lift) * p);
    holdWaveStance(); delay(delayTime);
  }
  delay(150);
```

```
  for (int i = 1; i <= shiftSteps; i++) {
    float p = (float)i / shiftSteps;
    float cx = x_shift - (x_shift * p);
    updateLegByXY(0,  1,  0, cx, -H);
    updateLegByXY(4,  5,  1, cx, y_brace_FR - ((-0.010) * p));
    updateLegByXY(8,  9,  2, cx, y_crouch_HL - (0.010 * p));
    updateLegByXY(12, 13, 3, cx, y_crouch_HR - (0.028 * p));
    delay(shiftDelay);
  }
  robot_state = 0;
}

void waveRightHand() {
  const int FR_THIGH = 4; const int FR_CALF = 5; const int FR_INDEX = 1;
  int shiftSteps = 40; int shiftDelay = 15;

  float x_shift = 0.025;
  float y_brace_FL = -H - 0.010;
  float y_crouch_HL = -H + 0.028;
  float y_crouch_HR = -H + 0.010;

  for (int i = 1; i <= shiftSteps; i++) {
    float p = (float)i / shiftSteps;
    updateLegByXY(0,  1,  0, x_shift * p, -H + (-0.010 * p));
    updateLegByXY(4,  5,  1, x_shift * p, -H);
    updateLegByXY(8,  9,  2, x_shift * p, -H + (0.028 * p));
    updateLegByXY(12, 13, 3, x_shift * p, -H + (0.010 * p));
    delay(shiftDelay);
  }
  delay(400);

  float x_start = x_shift; float y_start = -H;
  float x_lift = x_shift + 0.020; float y_lift = -H + 0.040;
  float x_wave_front = x_lift + 0.020; float x_wave_back = x_lift - 0.020;
  float y_wave = y_lift;

  auto holdWaveStance = [&]() {
    updateLegByXY(0,  1,  0, x_shift, y_brace_FL);
    updateLegByXY(8,  9,  2, x_shift, y_crouch_HL);
    updateLegByXY(12, 13, 3, x_shift, y_crouch_HR);
  };

  int steps = 20; int delayTime = 15;

  for (int i = 0; i <= steps; i++) {
    float p = (float)i / steps;
```

```
    updateLegByXY(FR_THIGH, FR_CALF, FR_INDEX, x_start + (x_lift - x_start) * p, y_start +
(y_lift - y_start) * p);
    holdWaveStance(); delay(delayTime);
   }
   delay(120);

   for (int wave = 0; wave < 3; wave++) {
    for (int i = 0; i <= steps; i++) {
     updateLegByXY(FR_THIGH, FR_CALF, FR_INDEX, x_wave_back + (x_wave_front -
x_wave_back) * (float)i/steps, y_wave);
     holdWaveStance(); delay(delayTime);
    }
    for (int i = 0; i <= steps; i++) {
     updateLegByXY(FR_THIGH, FR_CALF, FR_INDEX, x_wave_front + (x_wave_back -
x_wave_front) * (float)i/steps, y_wave);
     holdWaveStance(); delay(delayTime);
    }
   }

   for (int i = 0; i <= steps; i++) {
    float p = (float)i / steps;
    updateLegByXY(FR_THIGH, FR_CALF, FR_INDEX, x_lift + (x_start - x_lift) * p, y_lift +
(y_start - y_lift) * p);
    holdWaveStance(); delay(delayTime);
   }
   delay(150);

   for (int i = 1; i <= shiftSteps; i++) {
    float p = (float)i / shiftSteps;
    float cx = x_shift - (x_shift * p);
    updateLegByXY(0,  1,  0, cx, y_brace_FL - ((-0.010) * p));
    updateLegByXY(4,  5,  1, cx, -H);
    updateLegByXY(8,  9,  2, cx, y_crouch_HL - (0.028 * p));
    updateLegByXY(12, 13, 3, cx, y_crouch_HR - (0.010 * p));
    delay(shiftDelay);
   }
   robot_state = 0;
}

void raiseHeadPose() {
   int steps = 40; int delayTime = 15;
   for (int i = 1; i <= steps; i++) {
    float p = (float)i / steps;
    float y_front = -H - (0.012 * p);
    float y_back  = -H + (0.020 * p);
    updateLegByXY(0,  1,  0, 0.0, y_front);
    updateLegByXY(4,  5,  1, 0.0, y_front);
```

```
    updateLegByXY(8,  9,  2, 0.0, y_back);
    updateLegByXY(12, 13, 3, 0.0, y_back);
    delay(delayTime);
  }
}

void raiseButtPose() {
  int steps = 40; int delayTime = 15;
  for (int i = 1; i <= steps; i++) {
    float p = (float)i / steps;
    float y_front = -H + (0.015 * p);
    float y_back  = -H - (0.012 * p);
    updateLegByXY(0,  1,  0, 0.0, y_front);
    updateLegByXY(4,  5,  1, 0.0, y_front);
    updateLegByXY(8,  9,  2, 0.0, y_back);
    updateLegByXY(12, 13, 3, 0.0, y_back);
    delay(delayTime);
  }
}

void swayFrontBack() {
  const int FL_THIGH = 0;  const int FL_CALF = 1;  const int FL_INDEX = 0;
  const int FR_THIGH = 4;  const int FR_CALF = 5;  const int FR_INDEX = 1;
  const int HL_THIGH = 8;  const int HL_CALF = 9;  const int HL_INDEX = 2;
  const int HR_THIGH = 12; const int HR_CALF = 13; const int HR_INDEX = 3;

  float swayAmplitude = 0.025;
  int cycles = 3;
  int stepsPerCycle = 80;
  int delayTime = 12;

  for (int c = 0; c < cycles; c++) {
    for (int i = 0; i < stepsPerCycle; i++) {
      float angle = (float)i / stepsPerCycle * 2.0 * M_PI;
      float x_offset = swayAmplitude * sin(angle);

      updateLegByXY(FL_THIGH, FL_CALF, FL_INDEX, x_offset, -H);
      updateLegByXY(FR_THIGH, FR_CALF, FR_INDEX, x_offset, -H);
      updateLegByXY(HL_THIGH, HL_CALF, HL_INDEX, x_offset, -H);
      updateLegByXY(HR_THIGH, HR_CALF, HR_INDEX, x_offset, -H);

      delay(delayTime);
    }
  }
  standStill();
  current_time = 0;
  robot_state = 0;
```

```
}

void sitWaveSequence() {
  const int FL_THIGH = 0; const int FL_CALF = 1; const int FL_INDEX = 0;
  float x_start = 0.0; float y_start = -H - 0.012;
  float x_lift = 0.030; float y_lift = y_start + 0.040;
  float x_wave_front = x_lift + 0.020; float x_wave_back = x_lift - 0.020;
  float y_wave = y_lift;

  auto holdSitStance = [&]() {
    updateLegByXY(4,  5,  1, 0.0, -H - 0.012);
    updateLegByXY(8,  9,  2, 0.0, -H + 0.02);
    updateLegByXY(12, 13, 3, 0.0, -H + 0.02);
  };

  int steps = 20;
  int delayTime = 15;

  for (int i = 0; i <= steps; i++) {
    float p = (float)i / steps;
    float x = x_start + (x_lift - x_start) * p;
    float y = y_start + (y_lift - y_start) * p;

    updateLegByXY(FL_THIGH, FL_CALF, FL_INDEX, x, y);
    holdSitStance();
    delay(delayTime);
  }
  delay(120);

  for (int wave = 0; wave < 3; wave++) {
    for (int i = 0; i <= steps; i++) {
      float p = (float)i / steps;
      float x = x_wave_back + (x_wave_front - x_wave_back) * p;
      updateLegByXY(FL_THIGH, FL_CALF, FL_INDEX, x, y_wave);
      holdSitStance();
      delay(delayTime);
    }
    for (int i = 0; i <= steps; i++) {
      float p = (float)i / steps;
      float x = x_wave_front + (x_wave_back - x_wave_front) * p;
      updateLegByXY(FL_THIGH, FL_CALF, FL_INDEX, x, y_wave);
      holdSitStance();
      delay(delayTime);
    }
  }
  delay(100);
```

```
  for (int i = 0; i <= steps; i++) {
    float p = (float)i / steps;
    float x = x_lift + (x_start - x_lift) * p;
    float y = y_lift + (y_start - y_lift) * p;
    updateLegByXY(FL_THIGH, FL_CALF, FL_INDEX, x, y);
    holdSitStance();
    delay(delayTime);
  }
}

void getTrajectory(float phi, float dir, float &x, float &y, int legIndex) {
  float current_h_step = (legIndex == 2 || legIndex == 3) ? h_step_back : h_step_front;
  float current_S = (legIndex == 0 || legIndex == 2) ? S_L : S_R;
  float x_offset = 0;
  float actual_dir = -dir;

  if (dir < 0) {
    current_S *= 0.7;
    if (legIndex == 0 || legIndex == 2) { current_S *= 0.76; }
    if (legIndex == 1) { current_S *= 1.20; }
    if (legIndex == 3) { current_S *= 1.02; current_h_step += 0.001; }
    if (legIndex == 0 || legIndex == 1) { x_offset = actual_dir * 0.012; }
  }

  if (phi < 0.5) {
    float phi_norm = phi * 2.0;
    x = actual_dir * (-current_S / 2.0 + current_S * phi_norm) + x_offset;
    y = -H + current_h_step * sin(PI * phi_norm);
  }
  else {
    float phi_norm = (phi - 0.5) * 2.0;
    x = actual_dir * (current_S / 2.0 - current_S * phi_norm) + x_offset;
    y = -H;
  }
}

void solveIK(float x, float y, float &theta1_deg, float &theta2_deg) {
  float D2 = x * x + y * y;
  float D = sqrt(D2);

  if (D > (L1 + L2)) {
    x = x * ((L1 + L2) * 0.99 / D);
    y = y * ((L1 + L2) * 0.99 / D);
    D2 = x * x + y * y;
    D = sqrt(D2);
  }
```

```
  float cos_theta2 = (D2 - L1 * L1 - L2 * L2) / (2.0 * L1 * L2);
  cos_theta2 = constrain(cos_theta2, -1.0, 1.0);
  float alpha = atan2(y, x);
  float cos_beta = (D2 + L1 * L1 - L2 * L2) / (2.0 * L1 * D);
  cos_beta = constrain(cos_beta, -1.0, 1.0);
  float beta = acos(cos_beta);

  theta1_deg = (alpha - beta) * 180.0 / PI;
  theta2_deg = acos(cos_theta2) * 180.0 / PI;
}

void updateLeg(int thighID, int calfID, int legIndex, float phi, int walkDir) {
  float x, y, t1, t2;
  getTrajectory(phi, (float)walkDir, x, y, legIndex);
  solveIK(x, y, t1, t2);

  float thighDelta = (t1 - neutral_t1) * dirThigh[legIndex];
  float calfDelta  = (t2 - neutral_t2) * dirCalf[legIndex];

  int angleThigh = baseThigh[legIndex] + (int)thighDelta;
  int angleCalf  = baseCalf[legIndex]  + (int)calfDelta;

  updateServoLogic(thighID, angleThigh);
  updateServoLogic(calfID,  angleCalf);
}

void updateLegByXY(int thighID, int calfID, int legIndex, float x, float y) {
  float t1, t2;
  solveIK(x, y, t1, t2);

  float thighDelta = (t1 - neutral_t1) * dirThigh[legIndex];
  float calfDelta  = (t2 - neutral_t2) * dirCalf[legIndex];

  int angleThigh = baseThigh[legIndex] + (int)thighDelta;
  int angleCalf  = baseCalf[legIndex]  + (int)calfDelta;

  updateServoLogic(thighID, angleThigh);
  updateServoLogic(calfID,  angleCalf);
}
```